让 AI 帮你创建 Skill

- 来源: https://class.imooc.com/lesson/5288#mid=105141
- 阶段: 阶段五: 【能力封装】可复用技能
- 周: 第10周 Skill：可复用技能封装
- 模块: frontend-design实战
- 课节: 图文 1-1 让 AI 帮你创建 Skill

1-1 让 AI 帮你创建 Skill

让 AI 帮你创建 Skill

> [!TIP] 阅读指南 上一节我们学习了 Skill 的扩展能力——scripts、references、assets，理解了 Skill 如何与工具和大模型协作。本节转向一个更实际的问题：怎么创建自己的 Skill。 在开始之前，搞清楚一个事实：创建 Skill 不只有「手写」一条路。本节将介绍三种方式——从最笨到最聪明，然后把最聪明的方式亲手走一遍。你不需要记住所有细节，只需要知道：以后创建 Skill，优先用第三种方式。

创建 Skill 的三种方式

- 方式一：最原始的方式是手写 Skill——打开编辑器，一行行地敲 YAML Frontmatter，一条条地写指令规则。理论上这最灵活，可以精确控制每一个细节。

但现实中，几乎没有人在 AI 工具里手写一个几百行的 Skill 文件。SKILL.md 涉及 name、description 的措辞规范、Markdown 的结构组织、触发关键词的嵌入位置等多个容易出错的点，手写不仅耗时间，还常常因为一个字段的疏漏导致 Skill 无法触发。

这就好比手写汇编语言——不是不能，而是没人会这么做。

- 方式二：AI 辅助生成，人工修改打磨。给大模型一段需求描述，它吐出 SKILL.md，再做调整。这种方式比手写快得多，但有一个常见问题：大模型在不受约束的情况下生成的 Skill 往往不够规范。

description 写得太泛化，导致不知道该在什么场景触发；指令结构松散，关键规则埋在一大段文字里，执行效果打折扣。更麻烦的是，有些生成出来的 Skill 看似完整，实际缺少触发必需的字段或关键词，装上去就石沉大海——从不触发，也不知道哪里出了问题。

- 方式三：目前最推荐的方式，是使用 Anthropic 官方出品的 skill-creator——一个专门用来生成 Skill 的 Skill。它的视角和前面两种截然不同：skill-creator 不是在「帮你写一段 Markdown」，而是在「按照 Skill 的标准工程流程，引导产出一个规范的 Skill 包」。

它内置了 YAML 结构校验、description 触发测试、渐进加载设计等最佳实践，生成出来的 Skill 不会出现触发失败或结构残缺这类低级问题。更妙的是，使用门槛极低——在 Qoder 的对话里描述需求，它就会引导一步步完成。

实战：用 Skill Creator 创建第一个 Skill

说了这么多，来实际操作一遍。

获取 Skill Creator

skill-creator 是 Anthropic 官方出品的 Skill 工程工具，托管在 GitHub 上。安装方式非常简单——直接把仓库地址发给 Qoder，让它自动下载：

```
请帮我安装 https://github.com/anthropics/skills/tree/main/skills/skill-creator

1
```

Qoder 会自动完成克隆、放置和加载，全程不需要离开对话窗口。

安装后输入下面的指令，确认它已经就绪：

```
列出当前可用的 Skill

1
```

输出中应能看到 skill-creator。确认激活后，直接在对话中描述你想创建的 Skill。skill-creator 会接管后续的引导流程。

一个简单的例子：Commit Message 格式化器

假设你所在的团队想统一 Git commit message 的格式。与其靠口头约定然后在 code review 时反复提醒，不如直接设计一个团队共用的 Skill。所有人都可以参与讨论：type 列表里包含哪些、scope 用组件名还是模块名、description 用中文还是英文。一旦达成共识，Skill 就把这些约定固化下来了——不需要任何额外的沟通成本，每个人用起来结果都一样。这就用到了 Skill 的核心价值：不是教每个人记一套规范，而是将规范自己做成了工具。

在 Qoder 对话框中输入：

```
使用 skill-creator，帮我创建一个 Skill，功能如下：

我们团队需要统一 Git commit message 的格式。接收一段变更描述，输出标准化的 commit message。
格式为：type(scope): description
团队约定的 type 列表：fix/feat/chore/docs/refactor/test/style。
description 用祈使句，首字母小写，句尾不加句号。
如果有 body，用空行隔开；有关联 issue 则在 footer 中标注。

触发场景：用户说「加个 commit」「写提交信息」「记录这次变更」等。
```

skill-creator 收到描述后会进入引导流程。它会逐项确认功能边界，引导你做出一系列团队决策——比如 type 列表要不要包含 perf 和 ci？scope 的命名规范用什么粒度？description 用中文还是英文？这些正是团队需要对齐的地方，skill-creator 帮你逐个梳理清楚。

引导完成后，skill-creator 会输出一份完整的 SKILL.md。将这个文件保存到 Qoder 的项目技能目录：

```
你的项目/
  └── .qoder/
        └── skills/
              └── commit-formatter/
                    └── SKILL.md
```

保存后重启 Qoder（或输入「重新扫描 Skill」），Skill 即生效。

> [!TIP] 建议你自己打开 Qoder 跑一遍这个流程，感受 skill-creator 的引导过程。如果不想执行，也可以直接翻阅本书配套的样本文件，查看执行结果： samples/chapter10/commit-formatter/SKILL.md 本书中的这个示例就是按上述对话流程真实跑完后的输出产物。

■ 学点英语

中文	English	音标	说明
技能创建器	skill-creator	/skɪl kriːˈeɪtər/	Anthropic官方出品的Skill工程工具，按标准流程引导用户创建规范的Skill
提交信息	Commit Message	/kəˈmɪtˈmesɪdʒ/	Git提交的标准化描述，格式为 type(scope): description
元数据标记语言	YAML	/ˈjæml/	SKILL.md顶部元数据区的编写语言，定义name和description字段
标记语言	Markdown	/ˈmɑːrkdaʊn/	SKILL.md正文区的编写语言，用于描述Skill的指令和规范

下一节预告

学会了让 AI 生成 Skill，你已经具备了扩展 AI 能力的核心技能。但到目前为止，我们使用的都是独立 Skill——一个 Skill 只做一件事。

接下来，我们将挑战一个更真实的场景：用 Skill 来指挥另一个 Skill。具体来说，我们自己编写一个编排 Skill，让它驱动 frontend-design（专业 UI 设计技能），生成一个数据充实、支持列表和详情切换的个人技能可视化主页。

图片

